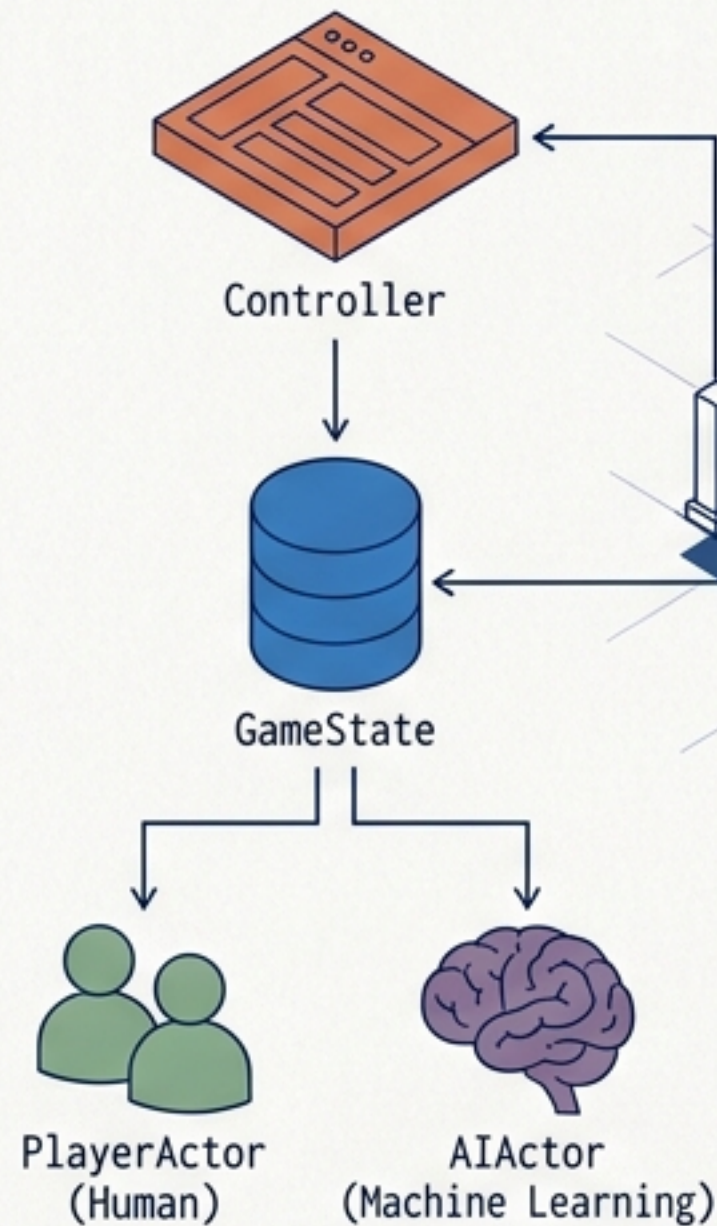


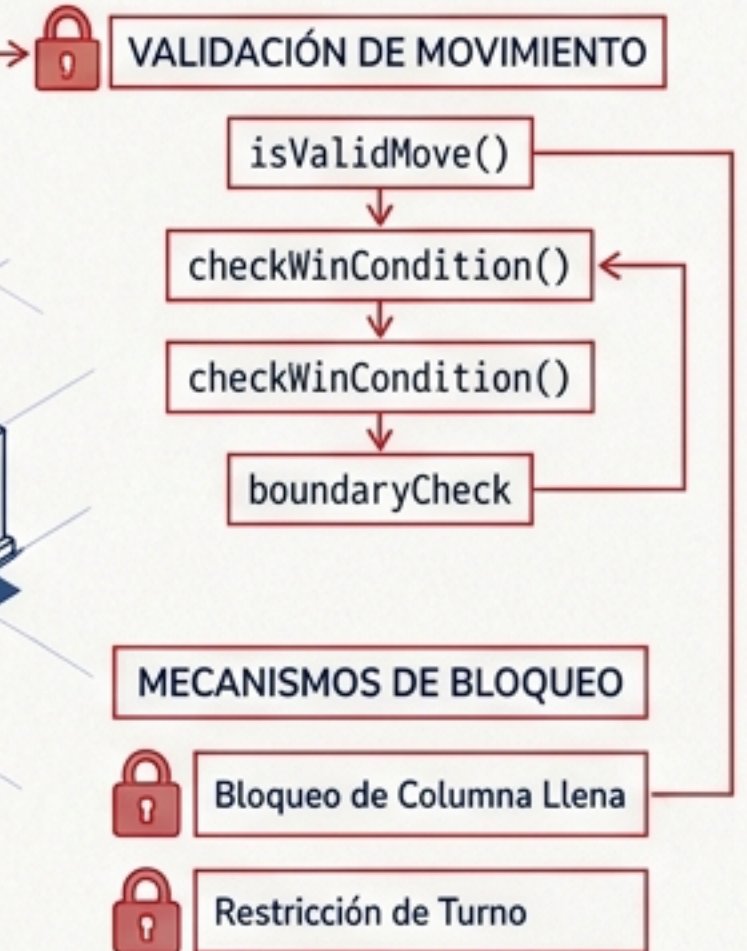
ARQUITECTURA DE CÓDIGO: DISEÑANDO EL 4 EN RAYA

De los principios sólidos a la ejecución polimórfica en C#

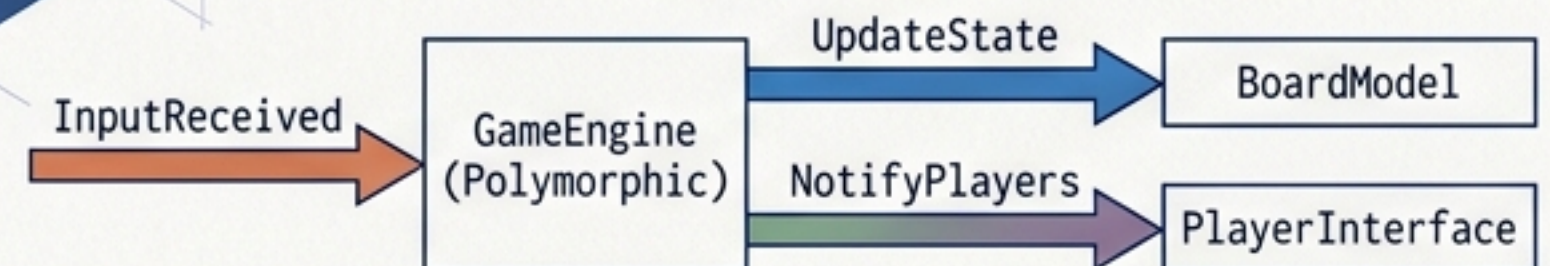
COMPONENTES DEL SISTEMA



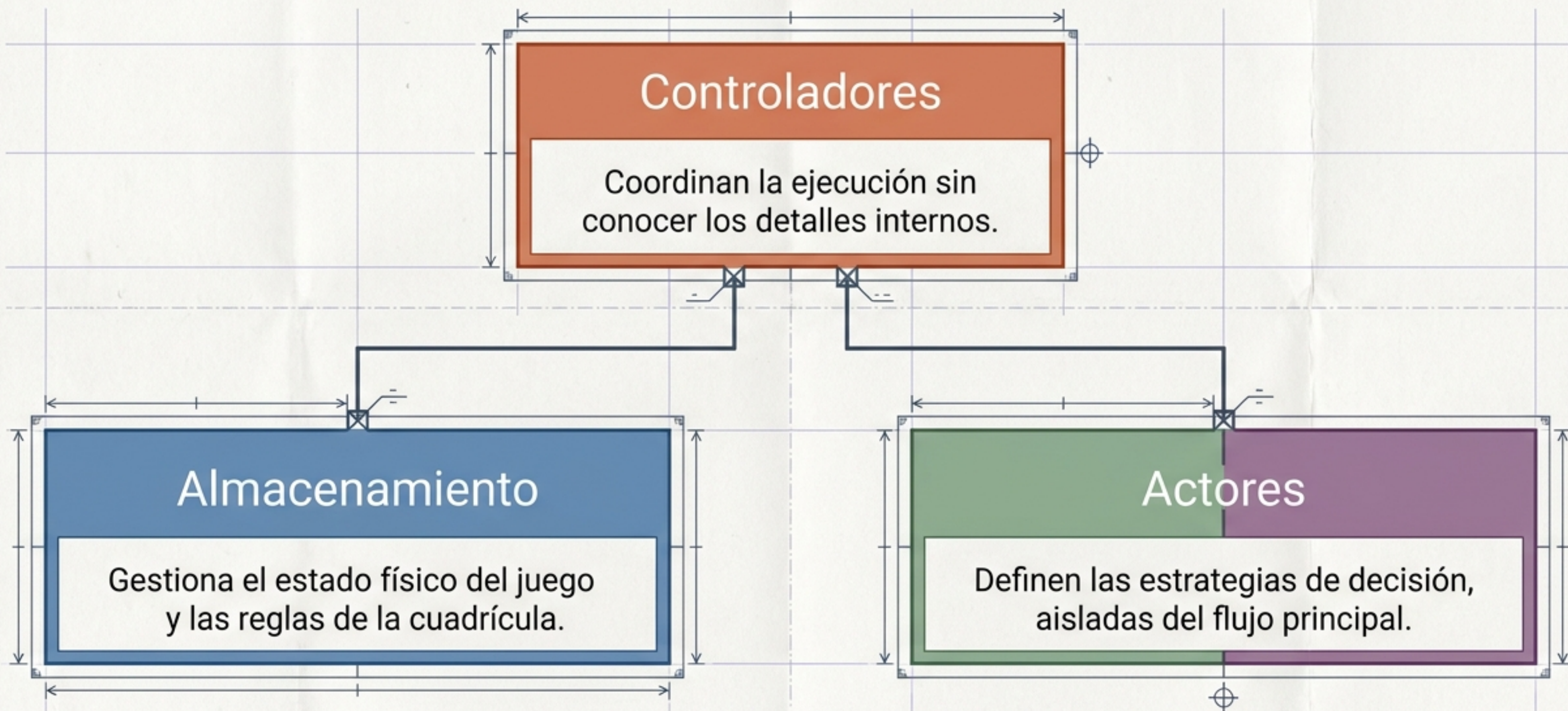
REGLAS SINTÁCTICAS Y LÓGICAS



FLUJO DE DATOS Y POLIMORFISMO

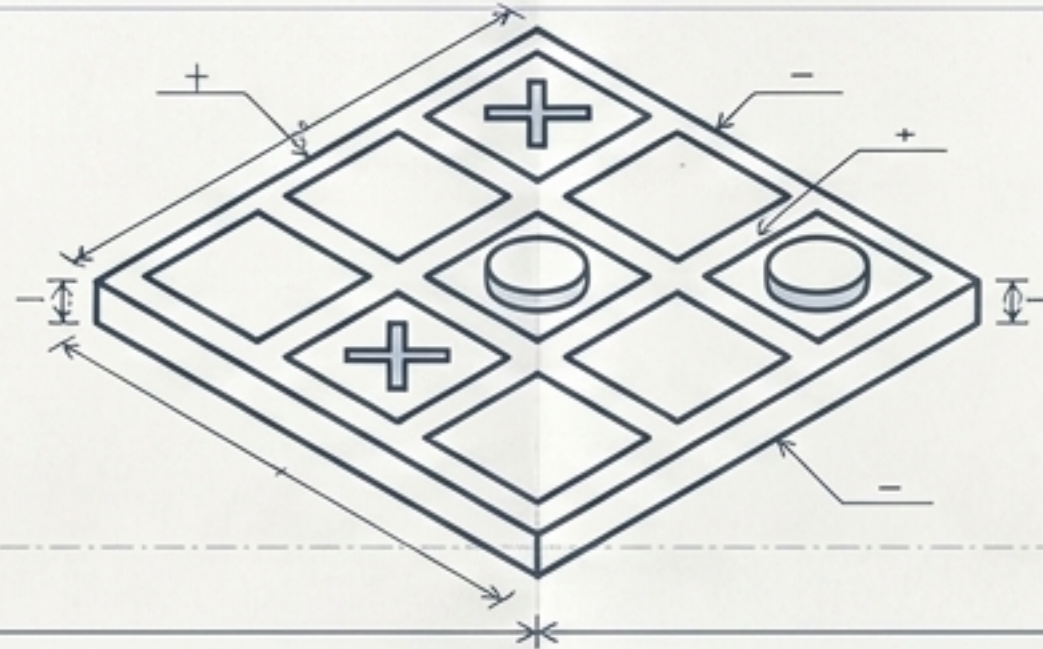


El Mapa del Sistema: Separación por Módulos



El Principio de Responsabilidad Única (SRP) en Acción

La clase **Tablero** es experta en la cuadrícula, pero ignorante del juego.



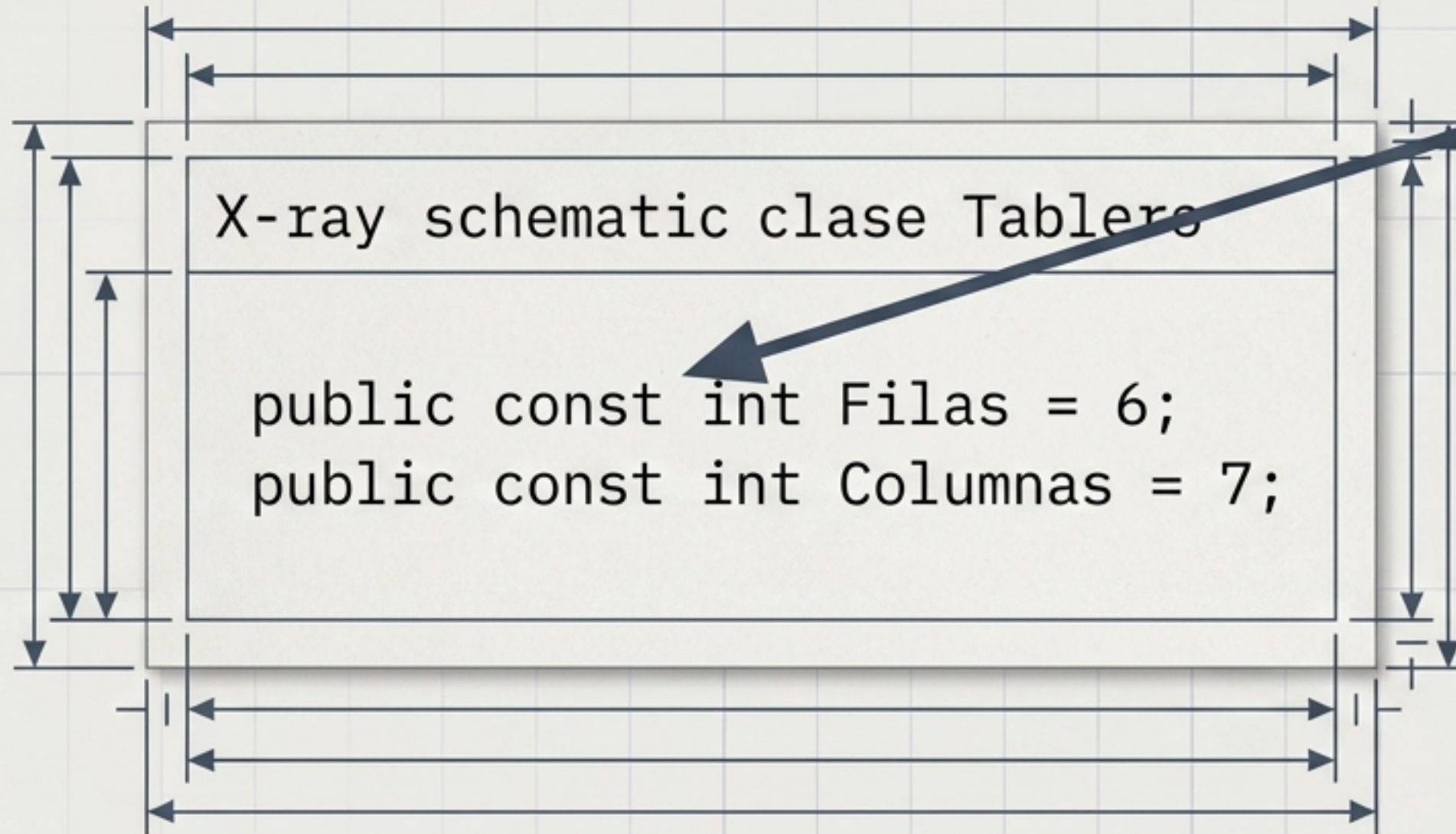
Lo que HACE

- ✓ Almacenar las casillas.
- ✓ Colocar o retirar fichas.  
- ✓ Comprobar si hay victoria. 
- ✓ Reiniciar la cuadrícula. 

Lo que NO HACE

- ✗ No decide de quién es el turno. 
- ✗ No pide datos por teclado. 
- ✗ No anuncia a los ganadores. 

Definiendo los Límites Físicos

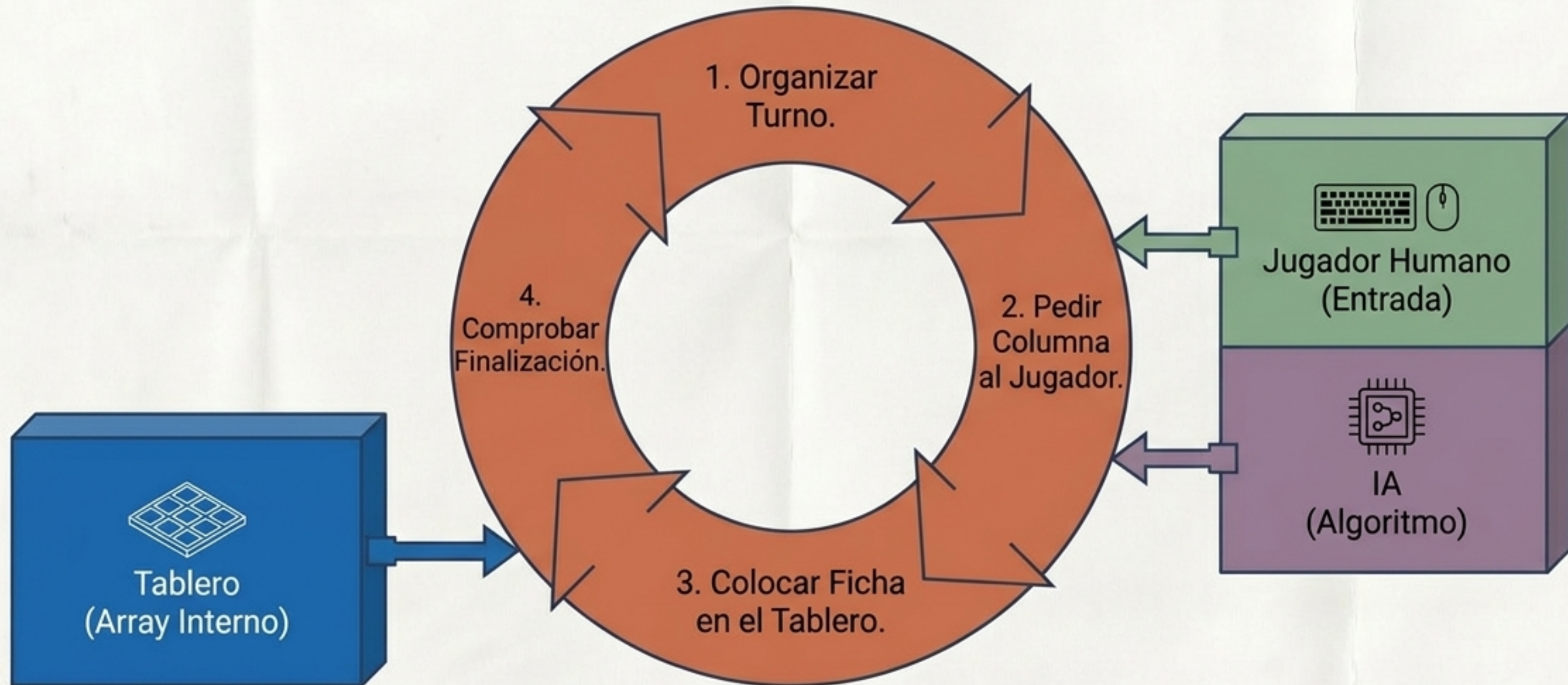


const (Constante)

Un tablero clásico siempre tiene estas dimensiones. Al hacerlas públicas y constantes, otras clases pueden conocer los límites del escenario de forma segura, sin riesgo de alterarlos durante la partida.

El Motor Central: El Bucle de Juego

La clase **Juego** es un orquestador ciego. No sabe cómo funciona el array interno del tablero, ni qué algoritmo usa la IA. Simplemente hace girar la rueda.



Protegiendo los Actores del Motor

X-ray schematic clase Juego

```
private readonly Tablero tablero;  
private readonly Jugador jugadorHumano;  
private readonly Jugador jugadorOrdenador;
```

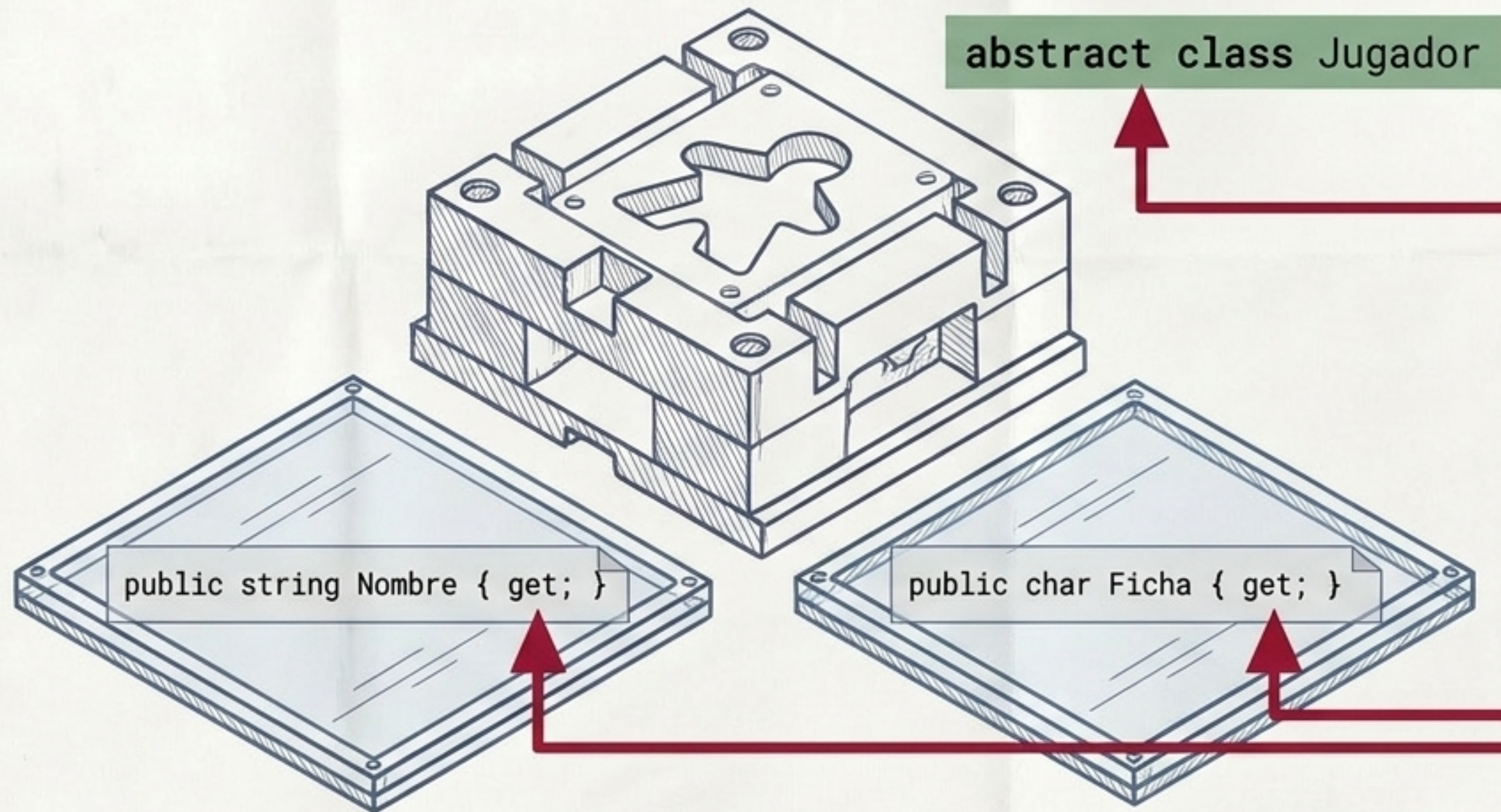


readonly (Solo Lectura)

Estos objetos se instancian una sola vez al arrancar el motor (en el constructor). Declararlos readonly garantiza arquitectónicamente que no sean sustituidos a mitad de la partida.

El Molde de Participante: Clases Abstractas

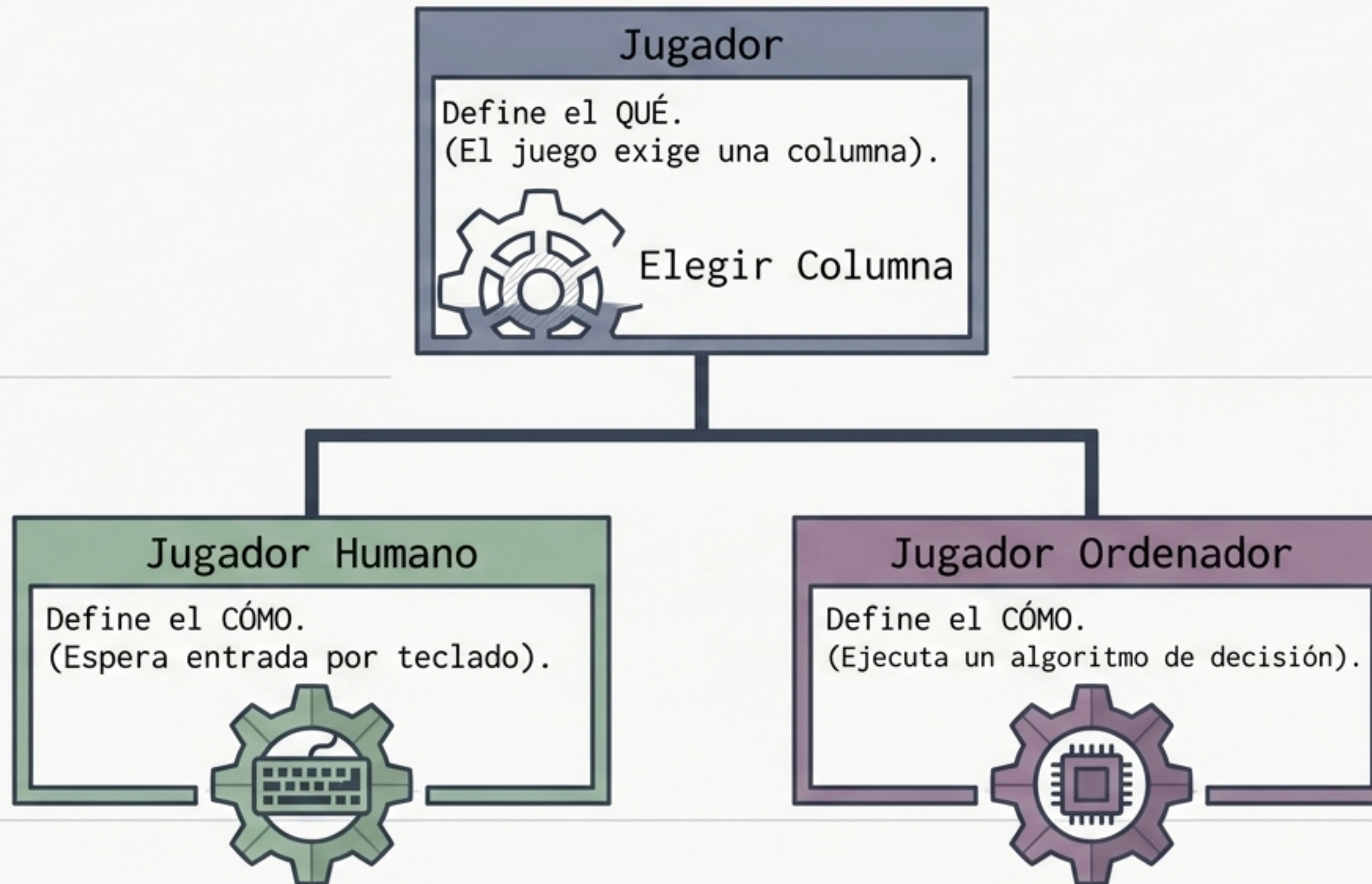
No tiene sentido instanciar un Jugador genérico en el vacío; debe ser un humano o una máquina.



abstract: Obliga a crear tipos concretos. Actúa como un contrato arquitectónico.

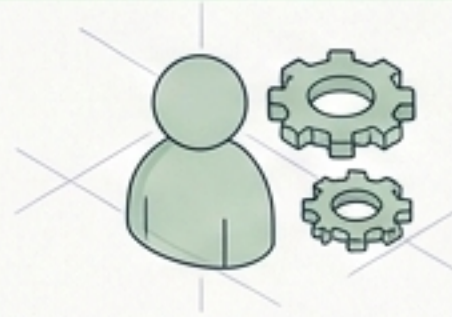
get (Sin set): La identidad del jugador y su ficha son inmutables tras su creación. El diseño **prohíbe cambios accidentales** durante el juego.

Herencia y Contratos de Comportamiento



Anatomía de la Inicialización: Humano vs IA

Jugador Humano

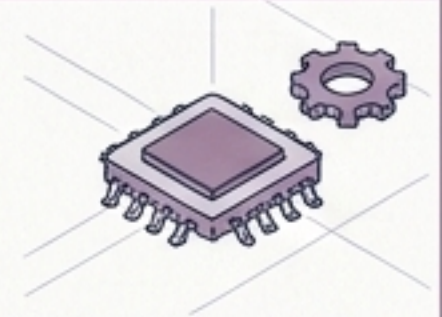


```
public JugadorHumano(...)  
    : base(nombre, ficha)
```

Usa `base()` para delegar la construcción del nombre y la ficha a la clase padre. No requiere estado adicional.



Jugador Ordenador

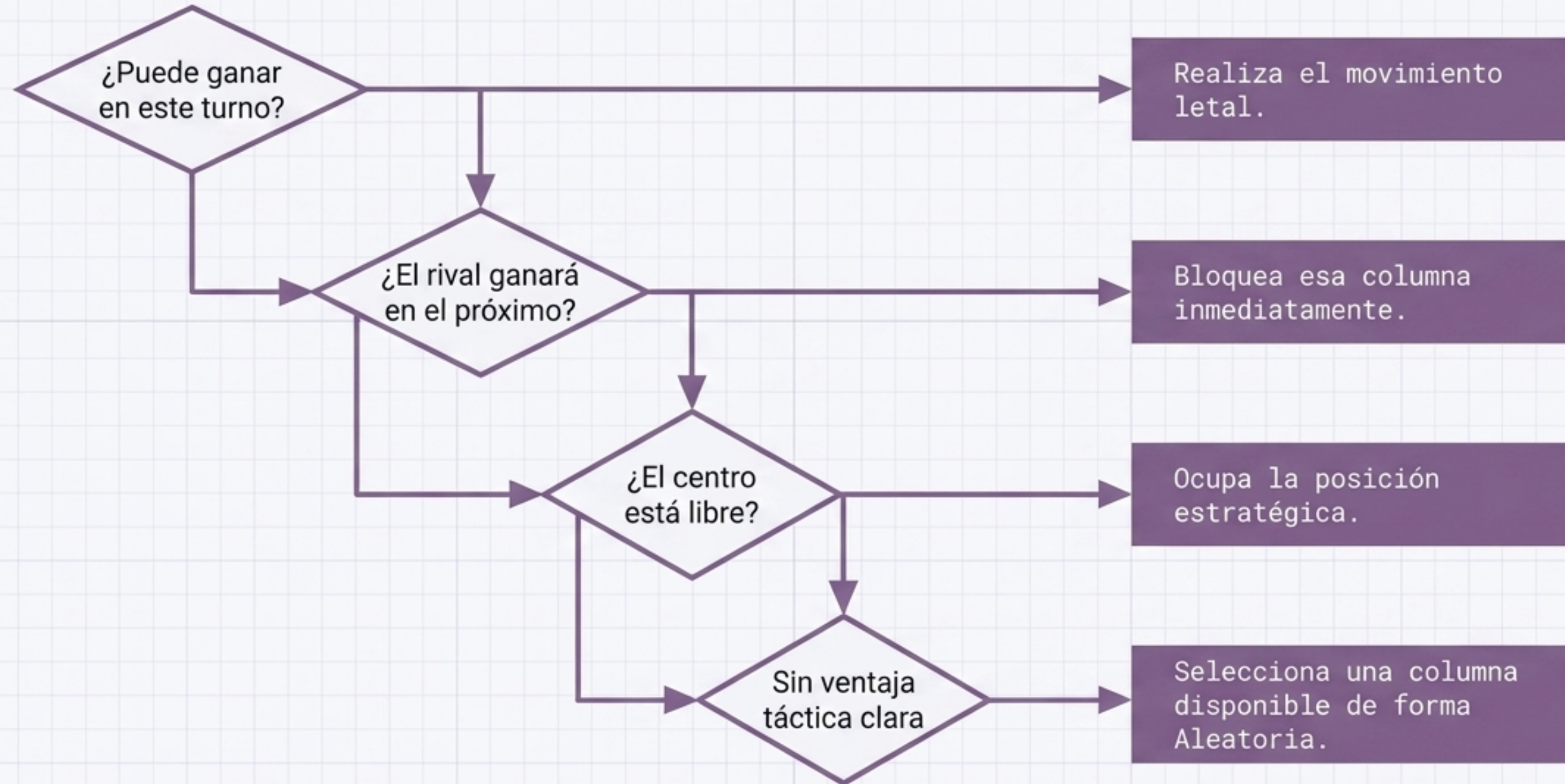


```
private readonly Random random;
```

Reutiliza un único generador `Random` para toda la partida. Crear uno nuevo en cada turno produciría secuencias repetidas debido a la velocidad de ejecución.

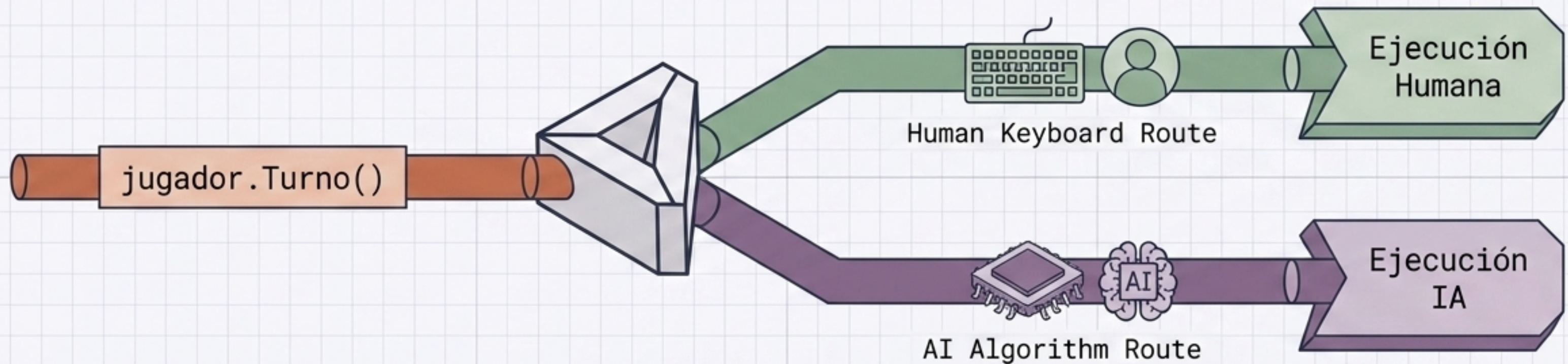


El Cerebro Sintético: Árbol de Decisión



La Magia del Polimorfismo

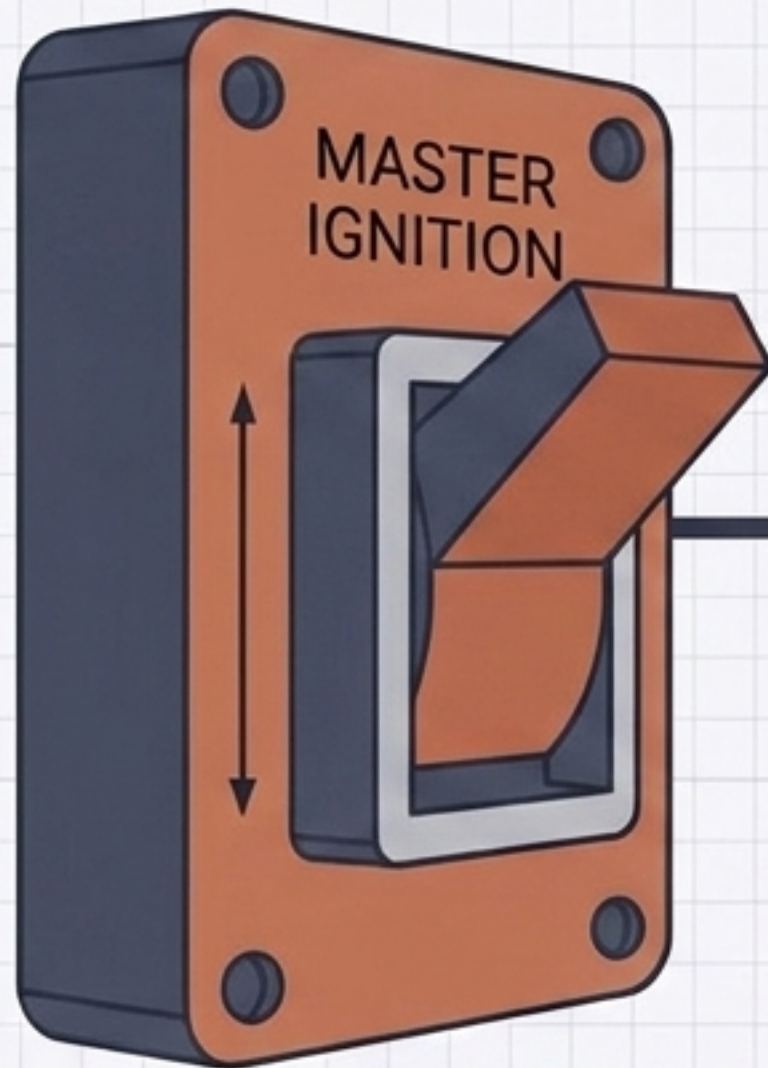
La clase Juego solo posee una variable de tipo Jugador. Cuando el motor invoca el turno, ignora por completo quién está al otro lado.



La ejecución se bifurca de manera invisible hacia el ser humano o **hacia la inteligencia artificial**. Esta es la **máxima expresión** del bajo acoplamiento: el coordinador organiza, los objetos actúan.

El Punto de Entrada: Encendiendo el Motor

Contexto: La clase Program



static: Su única responsabilidad es arrancar la aplicación. Al ser estática, la máquina virtual no necesita construir un objeto **Program**; simplemente gira la llave directamente.

```
static class Program {  
    private static void Main() { ... }  
}
```

El Toque Final: Configura el entorno (**Console.Title = "4 en raya"**) e invoca al coordinador maestro, dando vida a los planos arquitectónicos.